To write in the current log use the `logger.(debug|info|warn|error|fatal)` method from within a controller, model or mailer:

```
logger.debug "Person attributes hash: #{@person.attributes.inspect}"
logger.info "Processing the request..."
logger.fatal "Terminating application, raised unrecoverable error!!!"
```

Here's an example of a method instrumented with extra logging:

```
class ArticlesController < ApplicationController
  # ...

  def create
    @article = Article.new(params[:article])
    logger.debug "New article: #{@article.attributes.inspect}"
    logger.debug "Article should be valid: #{@article.valid?}"

    if @article.save
      flash[:notice] =  'Article was successfully created.'
      logger.debug "The article was saved and now the user is going to
be redirected..."
      redirect_to(@article)
    else
      render action: "new"
    end
  end

  # ...
end
```

Here's an example of the log generated when this controller action is executed:

```
Processing ArticlesController#create (for 127.0.0.1 at 2008-09-08 11:52:54) [POST]
  Session ID:
BAh7BzoMY3NyZl9pZCIlMDY5MWU1M2I1ZDRjODBlMzkyMWI1OTg2NWQyNzViZjYiCmZsYXNoSUM6J0FjdG
l
vbkNvbnRyb2xsZXI6OkZsYXNoOjpGbGFzaEhhc2h7AAY6CkB1c2VkewA=--
b18cd92fba90eacf8137e5f6b3b06c4d724596a4
  Parameters: {"commit"=>"Create", "article"=>{"title"=>"Debugging Rails",
 "body"=>"I'm learning how to print in logs!!!", "published"=>"0"},
 "authenticity_token"=>"2059c1286e93402e389127b1153204e0d1e275dd",
"action"=>"create", "controller"=>"articles"}
New article: {"updated_at"=>nil, "title"=>"Debugging Rails", "body"=>"I'm learning
how to print in logs!!!",
 "published"=>false, "created_at"=>nil}
Article should be valid: true
  Article Create (0.000443)   INSERT INTO "articles" ("updated_at", "title",
"body", "published",
 "created_at") VALUES('2008-09-08 14:52:54', 'Debugging Rails',
 'I''m learning how to print in logs!!!', 'f', '2008-09-08 14:52:54')
The article was saved and now the user is going to be redirected...
Redirected to # Article:0x20af760>
Completed in 0.01224 (81 reqs/sec) | DB: 0.00044 (3%) | 302 Found
[http://localhost/articles]
```

Adding extra logging like this makes it easy to search for unexpected or unusual behavior in your logs. If you add extra logging, be sure to make sensible use of log levels to avoid filling your production logs with useless trivia.

2.4 Tagged Logging

When running multi-user, multi-account applications, it's often useful to be able to filter the logs using some custom rules. `TaggedLogging` in Active Support helps in doing exactly that by stamping log lines with subdomains, request ids, and anything else to aid debugging such applications.